

Día 2 - Tienda de Regalos

1. Introducción y Problema El problema nos sitúa en una tienda de regalos con una base de datos corrupta. Tenemos rangos de IDs de productos (ej. "10-20") y debemos encontrar cuáles son inválidos y sumarlos. El reto tiene dos partes que cambian la definición de "inválido":

- **Parte A:** Un ID es inválido si está formado por una secuencia repetida **exactamente dos veces** (ej: 1212, o 11).
- **Parte B:** Un ID es inválido si la secuencia se repite **dos o más veces** (ej: 121212 y 111 también cuentan, además de los de la Parte A).

Las dos partes comparten todo: leer los rangos, expandirlos a IDs y sumar los inválidos. Lo único que cambia es la regla que decide si un ID es inválido. Por eso esa regla se modela como una estrategia intercambiable.

2. Arquitectura por capas He reorganizado el día en las mismas **cuatro capas** que el día 1, con las dependencias apuntando siempre hacia el dominio:

```
software.ulpgc.aoc.day02
├── model          (dominio puro, no depende de nadie)
│   └── IdRange
├── io             (frontera de entrada)
│   └── RangeLoader
├── control        (orquesta el caso de uso)
│   ├── Engine
│   ├── EngineBuilder
│   └── ValidationStrategies
└── application    (detalles y arranque)
    ├── ResourceRangeLoader
    └── a/Main02a, b/Main02b
```

Dirección de dependencias: application → control → (io + model) y io → model. El dominio (model) no importa nada del proyecto.

3. Explicación clase a clase **Capa model (dominio puro)**

- **IdRange** (*record*): representa un rango de IDs (p.ej. "10-20"). Tiene un constructor "traductor" que recibe el String sucio del fichero y lo convierte en dos long. Sabe expandirse a un flujo de números (getIds()) y filtrar los inválidos según una regla (getInvalidIds(validator)). No conoce ficheros, ni el motor, ni qué regla decide la validez → **alta cohesión**.

Capa io (frontera de entrada)

- **RangeLoader** (*interfaz*): define *qué* se necesita de la entrada (Stream<IdRange> loadAll()) sin atarse a *cómo* se lee. El arranque depende de esta abstracción, no del detalle de lectura.

Capa control (orquesta el caso de uso)

- **Engine** (*record*): el caso de uso. Recibe los rangos ya cargados y la estrategia de validación (LongPredicate), y en run() recorre cada rango, queda con los IDs inválidos

y los suma. No sabe de dónde vienen los rangos ni cómo se valida cada ID.

- **EngineBuilder** (*Patrón Builder, interfaz fluida*): construye el Engine paso a paso (`from(...).use(...).runner()`) y garantiza que nunca se cree incompleto (si falta la fuente o la estrategia, lanza excepción). Ya **no lee ficheros**: recibe los rangos ya cargados, así la construcción queda aislada del I/O.
- **ValidationStrategies** (*clase de utilidad*): agrupa las dos reglas de negocio (`PATTERN_A`, `PATTERN_B`) como constantes `LongPredicate` basadas en expresiones regulares. Su única razón para cambiar es que cambie la definición de ID “inválido”.

Capa application (detalles y arranque)

- **ResourceRangeLoader** (*implements RangeLoader*): el detalle de bajo nivel; lee un recurso del classpath, separa por comas, limpia y produce un `Stream<IdRange>`. Es intercambiable por otra fuente de datos.
- **Main02a / Main02b** (*composition root*): el único punto donde se eligen el cargador y la estrategia y se conectan con el Builder. La Parte A y la Parte B se diferencian solo en la estrategia inyectada.

4. Principios y diseños aplicados

- **Inversión de Dependencias (DIP)**: el Engine depende de la abstracción `LongPredicate` (recibe un `long` y dice si es válido), no de una regla concreta; el arranque depende de la interfaz `RangeLoader`, no de cómo se leen los datos.
- **Abierto/Cerrado (OCP)**: cambiar de la Parte A a la B (o añadir una regla nueva) es inyectar otra estrategia; el Engine queda cerrado a modificación.
- **Responsabilidad Única (SRP)**: `IdRange` solo entiende de rangos, Engine solo procesa, `ValidationStrategies` solo guarda las reglas, `ResourceRangeLoader` solo lee I/O, `EngineBuilder` solo ensambla.
- **Patrón Builder + interfaz fluida**: `EngineBuilder` arma el Engine paso a paso y valida que esté completo antes de crearlo.
- **Segregación de Interfaces (ISP)**: `RangeLoader` expone un único método cohesivo (`loadAll`).
- **Inyección de Dependencias (DI) / Strategy**: el cargador y la estrategia se pasan desde fuera; el comportamiento se elige sin tocar el núcleo.
- **Bajo Acoplamiento y DRY**: el procesamiento es independiente de las reglas de validación, y la lectura de entrada está centralizada en una sola implementación reutilizable.

5. Conclusión Al usar Streams y expresiones regulares bajo estas capas, se evita la complejidad de los bucles anidados y se separa con claridad el procesamiento (Engine) de las reglas (`ValidationStrategies`) y del I/O (`ResourceRangeLoader`). El resultado es un sistema con bajo acoplamiento, fácil de probar (los tests construyen Engine e `IdRange` directamente, sin tocar ficheros) y preparado para crecer. Se verificó que el comportamiento se conserva: los ejemplos de ambas partes siguen dando los mismos resultados (suma 33 en la Parte A y 210 en la Parte B).